

Runtrack C – J02

Job 01

Dans un fichier `address.c`, implémenter la fonction **print_address** qui affiche l'adresse contenue dans le pointeur (en hexadécimal) :

```
void print_address(int *n);
```

Fonctions autorisées : `write()`

Job 02

Dans un fichier `increment.c`, implémenter la fonction **increment** qui incrémenter la valeur pointée :

```
void increment(int *n);
```

Fonctions utilisées : aucune.

Job 03

Dans un fichier `swap.c`, implémenter la fonction **swap** qui prend en paramètres deux pointeurs sur `int` et échange leur valeur :

```
void swap(int *a, int *b);
```



Fonctions autorisées : aucune.

Job 04

Dans un fichier `sum_array.c`, implémenter la fonction **sum_array** qui calcule la somme des éléments dans un tableau d'entier, sans utiliser la syntaxe avec les crochets (`arr[]`) :

```
int sum_array(int *arr, int n);
```

Fonctions utilisées : aucune.

Job 05

Dans un fichier `int_dup.c`, implémenter la fonction **int_dup** qui alloue un `int`, y copie la valeur passée en paramètre, et retourne le pointeur :

```
int *int_dup(int value);
```

Fonctions autorisées : `malloc()`

Job 06

Dans un fichier `divide.c`, implémenter la fonction **divide** qui prend en paramètre un pointeur sur `int` et divise la valeur de l'`int` par 2 :

```
void divide(int *val);
```

Fonctions autorisées : aucune.



Job 07

Dans un fichier `array_clone.c`, implémenter la fonction **array_clone** qui alloue un nouveau tableau d'int et y copie les n premières valeurs. S'il y a moins de valeurs que n, on s'arrête au total de valeurs dans arr :

```
int *array_clone(int *arr, int n);
```

Fonctions autorisées : `malloc()`

Job 08

Dans un fichier `array_delete.c`, implémenter la fonction **array_delete** qui libère la mémoire du tableau passé en paramètre :

```
void array_delete(int *arr);
```

Fonctions autorisées : `free()`

Job 09

Dans un fichier `my_strdup.c`, implémenter la fonction **my_strdup** qui prend en paramètre une chaîne de caractères, alloue la mémoire nécessaire pour une copie de cette chaîne, et la copie dans l'espace mémoire fraîchement alloué :

```
char *swap(char *str);
```

Fonctions autorisées : `malloc()`

Indice : vous pouvez vous inspirer de **my_strcpy** du J01...



Job 10

Dans un fichier `sort.c`, implémenter la fonction **`sort_int`** qui prend en paramètre un tableau d'ints (se terminant par un pointeur NULL) et qui devra les trier dans l'ordre croissant. Un tri à bulles est suffisant :

```
int *sort(int *array);
```

Fonctions autorisées : aucune.

Compétences visées

- C

Base de connaissances

- https://www.w3schools.com/c/c_getstarted.php
- <https://openclassrooms.com/fr/courses/19980-apprenez-a-programmer-en-c>
- <https://www.my-mooc.com/fr/mooc/c-programming-getting-started>